Sprint 15 Structured Notes: Testing Strategy — Functional, E2E, Usability, Compatibility, Performance, Security, A/B, Alpha/Beta, and Regression

0. Where You Are Before Sprint 15

Before Sprint 15, you should already understand these React and testing ideas:

- React apps are built from components.
- JSX describes what should appear on the screen.
- Props pass data from parent components to child components.
- State lets React remember changing data.
- Events let React respond to clicks, typing, and form submissions.
- Forms can be tested by checking visible behavior.
- Effects, fetching, routing, Context, and performance ideas are already familiar.
- Sprint 14 introduced testing foundations.
- You know the basic idea of manual testing and automated testing.
- You know that a unit test checks one small piece of code.
- You know basic test syntax such as `test()`, `expect()`, and matchers.
- You know that React Testing Library encourages testing what the user sees and does.

Sprint 14 asked this question:

> Can I write a basic test?

Sprint 15 asks a more strategic question:

> Which test type should I use for this risk?

That is the main difference. Sprint 15 is not mainly about writing more `expect()` statements. It is about choosing the correct testing approach.

---

1. Sprint 15 Goal

The goal of Sprint 15 is to select the right test type for the right risk.

By the end of Sprint 15, you should be able to:

- Explain functional testing.

- Explain non-functional testing.
- Explain smoke testing.
- Use E2E testing for critical user flows.
- Explain usability testing.
- Explain compatibility testing.
- Explain performance testing.
- Explain security testing.
- Explain A/B testing.
- Explain alpha testing.
- Explain beta testing.
- Explain regression testing.
- Explain retesting.
- Match a test type to a realistic project risk.
- Avoid using E2E tests for simple pure functions.
- Create a test strategy for a React app.
- Build a simple QA Plan component.

Simple version:

> Sprint 15 teaches you how to stop testing randomly and start testing based on risk.

---

2. Big Mental Model

Different problems need different tests.

A beginner mistake is thinking:

> Testing means writing one kind of test for everything.

That is wrong.

A better mental model is:

```
Risk appears
↓
Choose the test type that best catches that risk
↓
Run the test
↓
Use the result to improve confidence
```

Example:

```
Small function may calculate wrong
→ Unit test

Search feature may not filter correctly
→ Functional test

User may not complete checkout
→ E2E test

Users may not understand the layout
→ Usability test

App may break on mobile Safari
→ Compatibility test

App may become slow under heavy traffic
→ Performance test

Private data may be exposed
→ Security test

Two button labels may perform differently
→ A/B test

New release may break old behavior
→ Regression test
```

Beginner rule:

> Do not ask only “Can I test this?” Ask “What risk am I testing?”

---

3. Test Choice Map

Use this map when choosing a test type:

Situation	Best test type
Small pure logic	Unit test
Feature behavior	Functional test
Critical user journey	E2E test
Confusing design or unclear UX	Usability test
Browser, device, OS, or network risk	Compatibility test

Situation	Best test type
Speed, load, or responsiveness risk	Performance test
Access, data protection, or attack risk	Security test
Recent code change may break old behavior	Regression test
A fixed bug needs to be checked again	Retesting
Two versions need to be compared with data	A/B test
App needs selected pre-release testers	Alpha testing
App needs limited real-user testing before full launch	Beta testing

Memory rule:

```
Logic problem → Unit
Feature problem → Functional
Journey problem → E2E
User confusion → Usability
Device/browser problem → Compatibility
Speed/load problem → Performance
Protection problem → Security
Old feature breaking → Regression
Fixed bug verification → Retesting
Version comparison → A/B
Pre-release selected testers → Alpha
Pre-release real users → Beta
```

---

4. Functional Testing

4.1 What Is Functional Testing?

Functional testing checks whether a feature works as expected.

Simple definition:

> Functional testing checks what the app does.

It answers this question:

> Does the feature behave correctly?

Examples:

```
Can the user log in?
Does search filter the course list?
Does clicking Enroll add the course?
Does the form show an error when the email is invalid?
Does the cart total update when quantity changes?
```

4.2 Functional Testing in React

In React, functional testing often checks visible behavior.

Example:

```
Render the Search component.
Type “React”.
Check that React courses appear.
Check that unrelated courses disappear.
```

This fits React Testing Library well because React Testing Library encourages testing from the user's point of view.

4.3 Unit Testing Is Part of Functional Testing

A unit test checks one small piece of behavior.

Example:

```
expect(formatPrice(4)).toBe("$4.00");
```

This is functional because it checks whether the function produces the correct result.

4.4 Smoke Testing Is Also a Basic Functional Check

Smoke testing is a quick basic check before deeper testing.

Example:

```
App opens.
Home page appears.
Login page opens.
Course list appears.
No major crash happens.
```

Smoke testing does not prove the whole app is correct. It only checks whether the app is basically alive.

Beginner rule:

> Functional testing checks whether features work.

---

5. Non-Functional Testing

5.1 What Is Non-Functional Testing?

Non-functional testing checks the quality of the app, not just whether individual features work.

Simple definition:

> Non-functional testing checks how well the app behaves.

It answers questions like:

```
Is the app fast?
Is the app usable?
Is the app secure?
Is the app reliable?
Does it work on different browsers?
Can it handle traffic?
Does it remain stable over time?
```

5.2 Functional vs Non-Functional Testing

Type	Main question	Example
Functional testing	Does it work?	Search returns matching courses
Non-functional testing	How well does it work?	Search remains fast with many courses

5.3 Beginner Comparison

Functional testing:

```
The login form lets a valid user log in.
```

Non-functional testing:

```
The login form responds quickly and securely under real traffic.
```

Beginner rule:

> Functional testing checks correctness. Non-functional testing checks quality.

---

6. Smoke Testing

6.1 What Is a Smoke Test?

A smoke test is a quick basic test that checks whether the main parts of the app are not obviously broken.

Simple definition:

> A smoke test checks whether the app is basically working before deeper testing.

The name comes from the idea of turning something on and checking whether smoke appears. In software, it means checking for obvious failure.

6.2 Smoke Test Example for a React App

```
Open the app.
Check that the Home page loads.
Open the Login page.
Open the Course List page.
Check that the main heading appears.
Check that no major crash screen appears.
```

6.3 What Smoke Testing Is Good For

Smoke testing is good for:

- Catching obvious crashes.
- Checking the app after deployment.
- Checking a build before deeper testing.
- Quickly confirming that the main pages still open.

6.4 What Smoke Testing Is Not Good For

Smoke testing is not enough for:

- Deep business logic.
- Full checkout flows.

- Security validation.
- Performance under load.
- Detailed form behavior.

Bad idea:

```
The smoke test passed, so the whole app is perfect.
```

Better idea:

```
The smoke test passed, so the app is stable enough for deeper testing.
```

Beginner rule:

> Smoke tests are fast safety checks, not complete proof.

---

7. Login Scenario Testing

Login is a common feature used to understand testing strategy.

A login feature can require many test types.

7.1 Functional Login Test

Question:

> Does login work when the user enters valid details?

Example:

```
Given the user enters a valid email and password
When the user submits the form
Then the dashboard appears
```

7.2 Validation Login Test

Question:

> Does the form reject bad input?

Example:

```
Given the email field is empty
When the user submits the form
Then an error message appears
```

7.3 Security Login Test

Question:

> Can someone bypass login or access private pages without permission?

Example:

```
Given the user is logged out
When the user opens /dashboard directly
Then the user is redirected to /login
```

7.4 E2E Login Flow

Question:

> Can the user complete the full login journey in the browser?

Example:

```
Open app
↓
Go to Login
↓
Type email
↓
Type password
↓
Submit
↓
Dashboard appears
```

Beginner rule:

> One feature can need several test types because it has several risks.

---

8. E2E Testing

8.1 What Is E2E Testing?

E2E means End-to-End.

Simple definition:

> E2E testing checks whether a real user flow works from start to finish.

It answers this question:

> Can a user complete an important journey through the app?

Examples:

```
Register → Log in → Open dashboard
Search course → Open details → Enroll
Add product to cart → Checkout → See confirmation
Open donation form → Fill details → Submit donation
```

8.2 Why E2E Testing Is Useful

E2E testing is useful because it checks many app layers together:

- Routes.
- Forms.
- Buttons.
- Browser behavior.
- API calls.
- Authentication flow.
- Page navigation.
- Real user actions.

8.3 Why E2E Testing Is Expensive

E2E tests can be expensive because they are:

- Slower than unit tests.
- More complex to set up.
- More fragile when UI changes.
- Harder to debug when they fail.
- Dependent on realistic test data.
- Dependent on browser automation.

Beginner rule:

Use E2E tests for critical flows, not every tiny detail.

8.4 Good E2E Test Candidates

Good E2E candidates:

```
Signup
Login
Checkout
Course enrollment
Payment flow
Donation flow
Password reset
Critical admin action
```

Poor E2E candidates:

```
formatPrice()
capitalizeName()
calculateTotal()
filterCourses()
small pure helper functions
```

Small pure functions should usually be tested with unit tests.

8.5 E2E Tools

Common E2E and regression testing tools include:

- Playwright.
- Cypress.
- Selenium.
- Puppeteer.

You do not need to master all of them now. The key idea is that these tools automate browser behavior.

8.6 Playwright-Style E2E Shape

```
import { test, expect } from "@playwright/test";

test("user can open products page", async ({ page }) => {
  await page.goto("/products");

  await expect(
```

```
    page.getByRole("heading", { name: "Products" })
  ).toBeVisible();
});
```

Read it like this:

```
Open /products.
Find a heading named Products.
Expect that heading to be visible.
```

8.7 E2E Example for Course Browser

```
User opens the app.
User logs in.
User searches for “React”.
User opens a React course.
User clicks Enroll.
User sees “Enrollment successful”.
```

Beginner rule:

> E2E tests protect the flows that would seriously hurt users or the business if they broke.

---

9. BDD Scenario Shape

BDD means Behavior-Driven Development.

Sprint 14 introduced BDD. Sprint 15 uses BDD-style scenarios to describe higher-level behavior.

BDD often uses this structure:

```
Given some starting situation
When the user does something
Then the expected result should happen
```

Example:

```
Given the user is logged in
When the user adds a product to cart
Then the cart count should increase
```

Another example:

```
Given the user is on the Course List page
When the user searches for “React”
Then only matching courses should be shown
```

BDD is useful because it describes behavior in plain language before or alongside code.

Beginner rule:

> Given/When/Then helps you describe user behavior clearly.

---

10. Usability Testing

10.1 What Is Usability Testing?

Usability testing checks whether real people can understand and use the app.

Simple definition:

> Usability testing checks whether the app is easy and clear for users.

It answers this question:

> Can users complete tasks without confusion?

Examples:

```
Can users find the search box?
Do users understand the Enroll button?
Can users finish checkout without help?
Do users notice validation errors?
Does the layout make sense?
Can users navigate the app easily?
```

10.2 Usability Testing Is Not Mainly About Code

A React component can be technically correct but still confusing.

Example:

```
The Enroll button works when clicked.
But users do not notice it.
```

That is not mainly a code bug. It is a usability problem.

10.3 Usability Testing Types

Type	Simple meaning	Example
Explorative testing	Test early ideas before final design	Show users a rough course page layout
Comparative testing	Compare two designs	Compare two course card layouts
Assessment testing	Check current usability	Watch users complete enrollment
Validation testing	Confirm final design works	Verify users can finish the final checkout flow

10.4 Usability Testing Example

```
Ask 5 users to find a React course and enroll.
Watch where they pause.
Do not explain the interface while they use it.
Record what confused them.
Use the results to improve the UI.
```

10.5 Usability Testing Tools

Common usability testing tools include:

- Loop11.
- Maze.
- Userbrain.
- UserTesting.
- UXTweak.

You do not need to memorize all tool details now. Just understand what they help with: observing real users and collecting usability feedback.

Beginner rule:

> Usability testing checks whether people can use the app, not just whether the code runs.

---

11. Compatibility Testing

11.1 What Is Compatibility Testing?

Compatibility testing checks whether the app works across different environments.

Simple definition:

> Compatibility testing checks whether the app works for different users on different setups.

It answers this question:

> Does this app still work outside my own computer and browser?

11.2 Compatibility Areas

Compatibility testing can include:

- Browser differences.
- Operating system differences.
- Hardware differences.
- Network differences.
- Mobile behavior.
- Tablet behavior.
- Desktop behavior.
- Backward compatibility.
- Forward compatibility.

11.3 Browser Compatibility

Examples:

```
Chrome
Firefox
Safari
Edge
Mobile Safari
Chrome Android
```

A layout may look correct in Chrome but break in Safari.

11.4 OS Compatibility

Examples:

```
Windows
macOS
Linux
iOS
Android
```

Some browser or font behavior can differ by operating system.

11.5 Hardware Compatibility

Examples:

```
Low-end phone
High-end phone
Small laptop
Large desktop monitor
Touchscreen device
Mouse and keyboard device
```

An app should not assume every user has a powerful machine or large screen.

11.6 Network Compatibility

Examples:

```
Fast Wi-Fi
Slow mobile data
Offline moment
High latency network
```

A data-heavy React app may feel fine on fast Wi-Fi and poor on slow mobile data.

11.7 Backward and Forward Compatibility

Backward compatibility means the app still works with older supported environments.

Example:

```
The app still works on an older supported browser version.
```

Forward compatibility means the app is likely to continue working with newer environments.

Example:

```
The app does not rely on fragile browser behavior that may break later.
```

11.8 Compatibility Checklist Example

```
Chrome desktop
Firefox desktop
Safari desktop
Chrome Android
Safari iPhone
Small screen
Large screen
Slow network
Keyboard navigation
Older supported browser version
```

Beginner rule:

> Your app is not fully checked until you test outside your favorite browser.

---

12. Performance Testing

12.1 What Is Performance Testing?

Performance testing checks speed, responsiveness, scalability, and stability.

Simple definition:

> Performance testing checks whether the app stays fast and stable under expected or difficult conditions.

It answers questions like:

```
Does the app load quickly?
Does the app respond quickly after clicks?
Can the backend handle many users?
Does the app remain stable over time?
What happens when traffic suddenly increases?
At what point does the system degrade?
```

12.2 Performance Testing vs React Optimization

Do not confuse performance testing with React memoization.

React optimization tools include things like:

```
useMemo
useCallback
React.memo
lazy loading
code splitting
```

Performance testing asks a broader question:

> How does the app or system behave under speed, load, or traffic pressure?

12.3 Load Testing

Load testing checks normal or expected traffic.

Simple definition:

> Load testing checks how the app performs under expected user activity.

Example:

```
1,000 users browse courses at the same time.
```

Question:

```
Can the app handle normal production traffic?
```

12.4 Stress Testing

Stress testing checks extreme traffic or heavy conditions.

Simple definition:

> Stress testing pushes the system beyond normal limits.

Example:

```
50,000 users hit the app at once.
```

Question:

```
How does the system fail under extreme pressure?
```

12.5 Soak Testing

Soak testing checks performance over a long time.

Simple definition:

> Soak testing checks whether the app stays stable during extended use.

Example:

```
Run the app under load for 8 hours.
```

Question:

```
Does the app slow down, leak memory, or become unstable over time?
```

12.6 Spike Testing

Spike testing checks sudden traffic jumps.

Simple definition:

> Spike testing checks how the app handles sharp increases in load.

Example:

```
100 users suddenly become 10,000 users.
```

Question:

```
Can the app handle sudden popularity or launch traffic?
```

12.7 Breakpoint or Capacity Testing

Breakpoint testing, also called capacity testing, increases load until the system starts to degrade.

Simple definition:

> Breakpoint testing finds the point where performance becomes unacceptable.

Example:

```
Increase users until response time becomes too slow.
```

Question:

```
What is the maximum load this system can handle before quality drops?
```

12.8 Performance Test Type Summary

Type	What it checks	Beginner example
Load test	Expected traffic	1,000 course browsers
Stress test	Extreme traffic	50,000 users at once
Soak test	Long-running stability	8 hours under load
Spike test	Sudden traffic jump	100 users → 10,000 users
Breakpoint/capacity test	Maximum safe limit	Increase load until response degrades

Beginner rule:

> Performance testing checks whether the app stays usable when real-world pressure appears.

---

13. Security Testing

13.1 What Is Security Testing?

Security testing checks whether the app protects data, permissions, and system behavior.

Simple definition:

> Security testing checks whether the app can resist threats and protect users.

It answers questions like:

```
Can private pages be accessed without login?
Can users access admin features without permission?
Can attackers inject harmful input?
Does the app expose sensitive errors?
Can the system be overwhelmed by traffic?
```

13.2 Security Principles

Sprint 15 introduces these security principles:

Principle	Simple meaning
Confidentiality	Private data stays private
Integrity	Data is not changed incorrectly
Authentication	The app knows who the user is
Authorization	The user can only access what they are allowed to access
Availability	The app remains usable
Non-repudiation	Important actions can be traced reliably

13.3 Confidentiality

Confidentiality means private information should not be exposed.

Example:

```
A user should not see another user's private profile data.
```

13.4 Integrity

Integrity means data should remain correct and protected from improper changes.

Example:

```
A user should not be able to change another user's course enrollment.
```

13.5 Authentication

Authentication means proving who the user is.

Example:

```
The app checks the user's email and password before login.
```

13.6 Authorization

Authorization means checking what the user is allowed to do.

Example:

```
A normal user can view courses.
Only an admin can delete courses.
```

Authentication and authorization are different.

```
Authentication = Who are you?
Authorization = What are you allowed to do?
```

13.7 Availability

Availability means the app remains usable.

Example:

```
The app should not become unavailable because of a flood of requests.
```

13.8 Non-Repudiation

Non-repudiation means important actions can be traced reliably.

Example:

```
If an admin deletes a course, the system records who did it and when.
```

13.9 Common Security Threats

Sprint 15 mentions these threats:

Threat	Simple meaning
XSS	Malicious script is injected into a page
SQL injection	Malicious input tries to change a database query
DoS	Attack traffic tries to make the app unavailable

13.10 XSS

XSS means Cross-Site Scripting.

Simple idea:

```
An attacker tries to make the page run harmful JavaScript.
```

Example risk:

```
A comment box displays user input unsafely.
```

13.11 SQL Injection

SQL injection happens when unsafe input affects a database query.

Simple idea:

```
An attacker types input that tries to manipulate a database command.
```

React alone does not solve SQL injection because SQL usually happens on the backend. But the frontend should still validate input and the backend must handle input safely.

13.12 DoS

DoS means Denial of Service.

Simple idea:

```
An attacker tries to overwhelm the app so real users cannot use it.
```

13.13 Security Testing Tools Categories

Security testing tools can fall into these categories:

Tool category	Simple meaning
SAST	Static Application Security Testing; checks code without running the app
DAST	Dynamic Application Security Testing; checks the running app
Penetration testing	Controlled attack-style testing to find weaknesses

You do not need to master these yet. For Sprint 15, understand the categories.

13.14 Security Test Examples for React Apps

```
Logged-out users cannot open /dashboard.
Normal users cannot open /admin.
Sensitive API errors are not shown directly to users.
Forms reject invalid input.
Private data does not appear in public pages.
Auth tokens are not displayed in the UI.
```

Important distinction:

```
Testing that login works → functional test
Testing that login cannot be bypassed → security test
```

Beginner rule:

> Security testing checks protection, not just feature success.

---

14. A/B Testing

14.1 What Is A/B Testing?

A/B testing compares two versions to see which one performs better.

Simple definition:

> A/B testing uses data to compare two versions of something.

It answers this question:

> Which version works better for a chosen goal?

14.2 A/B Testing Example

```
Version A button: “Start Learning”
Version B button: “View Courses”

Metric: click-through rate from home page to course list
```

If Version B gets more clicks, the team may choose Version B.

14.3 What A/B Testing Is Good For

A/B testing is good for comparing:

- Button text.
- CTA placement.
- Page headlines.
- Signup page layouts.
- Pricing page designs.
- Onboarding flows.

14.4 What A/B Testing Is Not For

A/B testing is not for checking security bugs or code correctness.

Bad:

```
Use A/B testing to find SQL injection issues.
```

Good:

```
Use A/B testing to compare two CTA button labels.
```

14.5 A/B Testing Tools

Common A/B or feature-flag experimentation tools include:

- GrowthBook.
- LaunchDarkly.

14.6 A/B Test Plan Example

```
Experiment:
Compare two CTA button labels.
```

```
Version A:
“Start Learning”

Version B:
“View Courses”

Success metric:
Click-through rate from home page to course list.

Decision rule:
Choose the version with the higher click-through rate after enough data is
collected.
```

Beginner rule:

> A/B testing compares product or design choices using data.

---

15. Alpha Testing

15.1 What Is Alpha Testing?

Alpha testing happens before public release with selected testers.

Simple definition:

> Alpha testing is early pre-release testing with a controlled group.

Example:

```
Your team or a small invited group tests the Course Browser before real users
get it.
```

15.2 Who Does Alpha Testing?

Alpha testers might be:

- Developers.
- QA testers.
- Internal team members.
- Selected trusted users.
- Stakeholders.

15.3 What Alpha Testing Is Good For

Alpha testing helps catch:

- Major bugs.
- Missing features.
- Confusing flows.
- Broken pages.
- Early usability problems.
- Release-blocking issues.

Beginner rule:

> Alpha testing happens before broad real-user release.

---

16. Beta Testing

16.1 What Is Beta Testing?

Beta testing happens with real users before the full public release.

Simple definition:

> Beta testing is limited real-user testing before full launch.

Example:

```
You release the Course Browser to 50 real learners before launching it to
everyone.
```

16.2 Alpha vs Beta Testing

Type	Who tests?	When?	Main purpose
Alpha testing	Selected internal or controlled testers	Earlier before release	Catch major issues early
Beta testing	Real users	Later before full release	Learn how the app behaves with real users

Short memory rule:

```
Alpha = early controlled testers
Beta = limited real users
```

16.3 What Beta Testing Is Good For

Beta testing helps find:

- Real-world bugs.
- Device-specific problems.
- Confusing user flows.
- Missing expectations.
- Performance issues under real usage.
- Feedback before full launch.

Beginner rule:

> Beta testing gives you real-world feedback before everyone gets the app.

---

17. Regression Testing

17.1 What Is Regression Testing?

Regression testing checks that old behavior still works after changes.

Simple definition:

> Regression testing checks that new changes did not accidentally break existing features.

It answers this question:

> Did something that used to work break again?

17.2 Why Regression Testing Matters

When you change code, you can accidentally break unrelated features.

Examples:

```
You change login and accidentally break protected routes.
You change course cards and accidentally break course search.
You update API data shape and accidentally break the dashboard.
You refactor checkout and accidentally break payment confirmation.
```

Regression testing protects old behavior while you add new behavior.

17.3 Regression Testing Example

```
Change made:
The course card layout was redesigned.

Regression tests to rerun:
Course list renders.
Search still filters courses.
Course detail links still work.
Enroll button still works.
Checkout flow still works.
```

17.4 Regression Testing Tools

E2E tools can also be used for regression tests, especially for repeated browser flows.

Common tools include:

- Playwright.
- Cypress.
- Selenium.
- Puppeteer.

Beginner rule:

> Regression testing protects existing behavior after code changes.

---

18. Retesting

18.1 What Is Retesting?

Retesting checks whether a known fixed bug is actually fixed.

Simple definition:

> Retesting checks a specific bug after a fix.

Example:

```
Bug:
Login fails even with the correct password.
```

```
Fix:
Update login logic.

Retesting:
Try the same valid login again and confirm it works.
```

18.2 Retesting vs Regression Testing

Type	Meaning	Example
Retesting	Check that a specific fixed bug is fixed	Login bug now works
Regression testing	Check that other existing features did not break	Signup, logout, dashboard, and protected routes still work

18.3 Beginner Example

```
Bug: Search did not show React courses.
Fix: Update search filter logic.

Retesting:
Search “React” again and confirm React courses appear.

Regression testing:
Also check empty search, course detail links, and enrollment still work.
```

Beginner rule:

> Retesting checks the known fix. Regression checks for unintended breakage.

---

19. Tools Mentioned in Sprint 15

You do not need to master every tool now, but you should know what category each belongs to.

Tool/category	Used for
Playwright	E2E and regression browser testing
Cypress	E2E and regression browser testing
Selenium	Browser automation and E2E testing
Puppeteer	Browser automation testing
Loop11	Usability testing

Tool/category	Used for
Maze	Usability testing
Userbrain	Usability testing
UserTesting	Usability testing
UXTweak	Usability testing
SAST tools	Static security checks in code
DAST tools	Security checks against a running app
Penetration testing	Controlled attack-style security testing
GrowthBook	A/B testing and experiments
LaunchDarkly	Feature flags and experiments

Beginner rule:

> Learn the test type first. Learn specific tools after you know why you need them.

---

20. Core Code Pattern: Playwright-Style E2E Test

```
import { test, expect } from "@playwright/test";

test("user can open products page", async ({ page }) => {
  await page.goto("/products");
  await expect(page.getByRole("heading", { name: "Products" })).toBeVisible();
});
```

Explanation

```
import { test, expect } from "@playwright/test";
```

This imports Playwright testing tools.

```
test("user can open products page", async ({ page }) => {
```

This creates a test. The `page` object represents a browser page.

```
await page.goto("/products");
```

This opens the `/products` route.

```
await expect(page.getByRole("heading", { name: "Products" })).toBeVisible();
```

This checks that a visible heading named `Products` appears.

Beginner rule:

> E2E code should read like user behavior.

---

21. Core Pattern: BDD Scenario Shape

```
Given the user is logged in
When the user adds a product to cart
Then the cart count should increase
```

Explanation

`Given` describes the starting situation.

`When` describes the user action.

`Then` describes the expected result.

Another example:

```
Given the user is on the Course List page
When the user types “React” in the search input
Then only React-related courses should appear
```

Beginner rule:

> If you cannot describe the behavior clearly in plain words, the test may be unclear too.

---

22. Course Browser Example: Full Test Strategy

Imagine you are building a React Course Browser with these parts:

```
Home page
Course list
Course detail page
Login page
Dashboard
Search
Enroll button
API data
Responsive layout
```

A good test strategy could look like this.

22.1 Unit Tests

Use unit tests for small pure logic.

```
formatPrice()
calculateCourseProgress()
filterCourses()
sortCoursesByLevel()
getCompletedLessonCount()
isValidEmail()
```

Example:

```
expect(formatPrice(4)).toBe("$4.00");
```

22.2 Functional Tests

Use functional tests for feature behavior.

```
Search filters the course list.
Empty search shows all courses.
Login form shows validation errors.
Enroll button changes UI after click.
Course detail page shows selected course data.
Dashboard shows enrolled courses.
```

22.3 E2E Flow

Use E2E for the most important journey.

```
User opens the app.
User logs in.
User searches for “React”.
User opens a course detail page.
User clicks Enroll.
User sees a success message.
User sees the course in the dashboard.
```

22.4 Compatibility Checks

Use compatibility testing for environment risks.

```
Chrome desktop
Firefox desktop
Safari desktop
Chrome Android
Safari iPhone
Small screen layout
Large screen layout
Keyboard navigation
Slow network behavior
```

22.5 Performance Checks

Use performance testing for speed and load risks.

```
Course list remains responsive with many courses.
Images are optimized.
Search does not freeze the UI.
Dashboard loads useful content quickly.
App gives feedback during slow actions.
API-heavy pages handle expected traffic.
```

22.6 Security Checks

Use security testing for access and data risks.

```
Logged-out users cannot access dashboard.
Normal users cannot access admin routes.
Private user data is not shown publicly.
Sensitive errors are not shown directly in the UI.
Invalid or unsafe input is handled safely.
```

22.7 Regression Triggers

Run regression tests after risky changes.

```
Any change to login.
Any change to routing.
Any change to course enrollment.
Any change to API response shape.
Any change to search logic.
Any change to shared auth state.
Any dependency upgrade affecting routing, forms, or tests.
```

22.8 A/B Test Plan

Use A/B testing for product/design comparisons.

```
Experiment:
Compare two CTA button labels.

Version A:
“Start Learning”

Version B:
“View Courses”

Success metric:
Click-through rate from home page to course list.

Decision rule:
Choose the version with the higher click-through rate after enough data is
collected.
```

Beginner rule:

> A test strategy is a map of risks and the tests that protect against them.

---

23. Common Beginner Mistakes

Mistake 1: Using E2E for Everything

Bad:

```
Use E2E to test formatPrice() returns "$4.00".
```

Why this is bad:

`formatPrice()` is small pure logic. A unit test is faster and simpler.

Better:

```
Use a unit test for formatPrice().
```

Mistake 2: Using Unit Tests for Full User Journeys

Bad:

```
Use unit testing to test full checkout across browser and payment page.
```

Why this is bad:

Checkout is a full user flow involving many parts of the app.

Better:

```
Use E2E testing for the full checkout flow.
```

Mistake 3: Using A/B Testing for Security Bugs

Bad:

```
Use A/B testing to find SQL injection issues.
```

Why this is bad:

SQL injection is a security risk, not a product experiment.

Better:

```
Use security testing to check SQL injection risk.
```

Mistake 4: Using Usability Testing for Server Load

Bad:

```
Use usability testing to measure server load under 10,000 users.
```

Why this is bad:

10,000 users is a load and performance risk.

Better:

```
Use performance/load testing to measure server load under 10,000 users.
```

Mistake 5: Thinking Passing Tests Mean Zero Bugs

Bad:

```
All tests passed, so there are definitely no bugs.
```

Better:

```
All tests passed for the behaviors we checked.
Other untested risks may still exist.
```

Beginner rule:

> Tests increase confidence. They do not magically prove perfection.

---

24. Debug Task: Fix the Broken Strategy

Broken strategy:

```
- Use E2E to test formatPrice() returns "$4.00".
- Use unit testing to test full checkout across browser and payment page.
- Use A/B testing to find SQL injection issues.
- Use usability testing to measure server load under 10,000 users.
```

Correct strategy:

```
- Use a unit test to test formatPrice() returns "$4.00".
- Use E2E testing to test full checkout across browser and payment page.
- Use security testing to check SQL injection risk.
- Use performance/load testing to measure server load under 10,000 users.
```

Why:

```
formatPrice() is small pure logic → unit test
checkout is a critical user journey → E2E test
SQL injection is a security risk → security test
10,000 users is a load/scalability risk → performance test
```

---

25. Mini Project: QA Plan Component

25.1 Goal

Build a one-page React component that displays test categories for a React app.

This project is not a real automated test suite. It is a test strategy page.

The purpose is to practice matching test types to risks.

25.2 Component Code

```jsx
const qaCategories = [
  {
    type: "Unit Test",
    risk: "Small pure logic may return the wrong value",
    example: 'formatPrice(4) should return "$4.00"',
  },
  {
    type: "Functional Test",
    risk: "A feature may not behave correctly",
    example: "Search should filter courses by keyword",
```

```
  },
  {
    type: "Smoke Test",
    risk: "The app may be broken immediately after build or deploy",
    example: "Home, Login, and Course List pages should open without crashing",
  },
  {
    type: "E2E Test",
    risk: "A critical user journey may fail",
    example: "User logs in, selects a course, and enrolls",
  },
  {
    type: "Usability Test",
    risk: "Users may not understand the interface",
    example: "Users should find the enroll button without help",
  },
  {
    type: "Compatibility Test",
    risk: "The app may break on some devices or browsers",
    example: "Course cards should work on mobile Safari",
  },
  {
    type: "Performance Test",
    risk: "The app may become slow under load",
    example: "Course list should respond quickly with many users",
  },
  {
    type: "Security Test",
    risk: "Private data or private pages may be exposed",
    example: "Logged-out users cannot access the dashboard",
  },
  {
    type: "A/B Test",
    risk: "We may not know which design performs better",
    example: 'Compare "Start Learning" vs "View Courses"',
  },
  {
    type: "Alpha Test",
    risk: "Major issues may exist before public release",
    example: "Selected testers try the app before release",
  },
  {
    type: "Beta Test",
    risk: "Real users may find issues before full launch",
    example: "A small group of real learners uses the app before public launch",
  },
  {
    type: "Regression Test",
```

```
    risk: "New changes may break old behavior",
    example: "After changing course cards, rerun search and enroll tests",
  },
  {
    type: "Retesting",
    risk: "A known bug fix may not actually work",
    example: "After fixing login, test the same failed login case again",
  },
];

export default function QAPlan() {
  return (
    <main>
      <h1>QA Plan for Course Browser</h1>
      <p>
        This plan matches each test type to the risk it is designed to catch.
      </p>

      <section>
        {qaCategories.map((item) => (
          <article key={item.type}>
            <h2>{item.type}</h2>
            <p>
              <strong>Risk:</strong> {item.risk}
            </p>
            <p>
              <strong>Example:</strong> {item.example}
            </p>
          </article>
        ))}
      </section>
    </main>
  );
}
```

25.3 What This Component Practices

This component practices:

- Rendering a list with `map()`.
- Using stable keys.
- Structuring simple data.
- Explaining test types.
- Matching risks to test categories.
- Thinking like a QA planner.

Beginner rule:

A QA plan does not only say “write tests.” It says which risks each test protects.

---

26. Mandatory Practice Coding Block

26.1 Warm-Up Drills

Write one smoke test idea:

```
The app opens, the Home page heading appears, and the Course List page loads.
```

Write one E2E scenario:

```
User logs in, searches for “React”, opens a course, clicks Enroll, and sees a
success message.
```

Write one compatibility checklist item:

```
Course cards must display correctly on iPhone Safari and Chrome Android.
```

26.2 Guided Coding Task

Create a test strategy for your Course Browser.

Include:

```
Unit tests
Functional tests
One E2E flow
Compatibility checks
Regression triggers
```

Example answer:

```
Unit tests:
- formatPrice()
- calculateCourseProgress()
- filterCourses()

Functional tests:
```

```
- Search filters course list
- Empty search shows all courses
- Login form shows validation errors
- Enroll button changes UI after click

E2E flow:
- User logs in
- Searches for a course
- Opens course detail
- Clicks Enroll
- Sees success message

Compatibility checks:
- Chrome desktop
- Firefox desktop
- Safari mobile
- Small screen layout
- Keyboard navigation

Regression triggers:
- Any change to login
- Any change to routing
- Any change to course enrollment
- Any change to API response shape
```

26.3 Debug Task

Fix the test type labels in this broken strategy:

```
Strategy:
- Use E2E to test formatPrice() returns "$4.00".
- Use unit testing to test full checkout across browser and payment page.
- Use A/B testing to find SQL injection issues.
- Use usability testing to measure server load under 10,000 users.
```

Correct answer:

```
- Use unit testing to test formatPrice() returns "$4.00".
- Use E2E testing to test full checkout across browser and payment page.
- Use security testing to find SQL injection issues.
- Use performance/load testing to measure server load under 10,000 users.
```

26.4 Build Something Small

Build a one-page QA Plan component.

Requirements:

```
Show test categories.
Show the risk each category catches.
Show one example for each category.
Render the categories from an array.
Use a stable key.
```

26.5 Challenge Task

Add a fake A/B test plan for a CTA button.

Example:

```
Experiment:
Compare two CTA button labels.

Version A:
“Start Learning”

Version B:
“View Courses”

Success metric:
Click-through rate from home page to course list.

Decision rule:
Choose the version with the higher click-through rate after enough data is
collected.
```

26.6 Expected Output / Self-Check

You are successful when:

```
Every test type is matched to a realistic risk.
E2E is not overused for simple pure functions.
Security risks are matched to security testing.
```

```
Load and traffic risks are matched to performance testing.
A/B testing is used only for comparing versions.
Regression testing is used after changes.
Retesting is used for known bug fixes.
```

---

27. Revision Checkpoint Questions

Answer these without looking at the notes.

Question 1

What is the difference between functional and non-functional testing?

Expected answer:

```
Functional testing checks whether features work correctly.
Non-functional testing checks how well the app behaves, such as performance,
reliability, usability, compatibility, and security.
```

Question 2

Why are E2E tests usually reserved for critical flows?

Expected answer:

```
Because E2E tests are useful but slower, more complex, and more expensive to
maintain than smaller tests. They should protect important user journeys like
login, checkout, enrollment, or payment.
```

Question 3

Name three security testing principles.

Possible answer:

```
Confidentiality, integrity, authentication, authorization, availability, and
non-repudiation.
```

Any three are acceptable.

Question 4

What is the difference between retesting and regression testing?

Expected answer:

```
Retesting checks whether a specific fixed bug is fixed.
Regression testing checks whether recent changes accidentally broke other
existing features.
```

---

28. Sprint Retrospective Prompts

After practicing Sprint 15, answer these:

```
What felt easy?
What broke first?
What concept needs one more tiny app?
What would you refactor if this became a real project?
```

More specific Sprint 15 reflection questions:

```
Which test type was easiest to understand?
Which test type was easiest to confuse with another?
Did you overuse E2E testing anywhere?
Did you clearly separate security, performance, and usability risks?
Can your QA Plan explain why each test type exists?
```

---

29. Sprint 15 Checklist

You are ready to move on from Sprint 15 when you can:

- Explain functional testing.
- Explain non-functional testing.
- Explain smoke testing.
- Explain E2E testing.
- Explain why E2E tests are used for critical flows.
- Name common E2E tools such as Playwright, Cypress, Selenium, and Puppeteer.
- Explain usability testing.
- Name usability testing types: explorative, comparative, assessment, and validation.
- Explain compatibility testing.

- List compatibility areas: browser, OS, hardware, network, mobile, backward compatibility, and forward compatibility.
- Explain performance testing.
- Explain load testing.
- Explain stress testing.
- Explain soak testing.
- Explain spike testing.
- Explain breakpoint or capacity testing.
- Explain security testing.
- Explain confidentiality, integrity, authentication, authorization, availability, and non-repudiation.
- Recognize threats such as XSS, SQL injection, and DoS.
- Understand SAST, DAST, and penetration testing at a beginner level.
- Explain A/B testing.
- Explain alpha testing.
- Explain beta testing.
- Explain regression testing.
- Explain retesting.
- Correctly fix the broken strategy labels.
- Create a Course Browser test strategy.
- Build a QA Plan component.
- Avoid using E2E tests for simple pure functions.

---

30. One-Page Memory Summary

Sprint 15 is about testing strategy.

The main idea is:

```
Choose the test type based on the risk.
```

Functional testing checks whether features work.

Non-functional testing checks quality, such as performance, usability, compatibility, security, and reliability.

Smoke testing checks whether the app is basically alive.

E2E testing checks complete user journeys like login, checkout, enrollment, registration, or donation.

E2E tests are powerful but expensive, so reserve them for critical flows.

Usability testing checks whether real people can understand and use the app.

Compatibility testing checks browsers, operating systems, hardware, networks, mobile behavior, backward compatibility, and forward compatibility.

Performance testing checks speed, responsiveness, scalability, and stability.

Load testing checks expected traffic.

Stress testing checks extreme traffic.

Soak testing checks long-running stability.

Spike testing checks sudden traffic jumps.

Breakpoint or capacity testing finds when the system starts to degrade.

Security testing checks protection: confidentiality, integrity, authentication, authorization, availability, and non-repudiation.

Common security threats include XSS, SQL injection, and DoS.

Security tool categories include SAST, DAST, and penetration testing.

A/B testing compares two versions and uses data, such as click-through rate, to choose the better version.

Alpha testing uses selected testers before release.

Beta testing uses real users before full launch.

Regression testing reruns tests after changes to catch unintended breakage.

Retesting checks whether a known fixed bug is actually fixed.

Final memory rule:

```
Small logic → Unit test
Feature behavior → Functional test
Critical journey → E2E test
Basic app health → Smoke test
User confusion → Usability test
Browser/device risk → Compatibility test
Speed/load risk → Performance test
Access/data risk → Security test
Version comparison → A/B test
Early selected testers → Alpha test
Limited real users → Beta test
Recent change risk → Regression test
Known bug fix → Retesting
```

Shortest Sprint 15 summary:

> Sprint 15 teaches you to stop asking “Can I test this?” and start asking “What risk am I testing, and which test type fits that risk?”